Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

Nombres: María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz **Fecha:** 17 de Marzo de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie.

```
from google.colab import drive
drive.mount('/content/drive')

import os

ruta_carpeta = '/content/drive/MyDrive/PUJ - Maestría - Proyecto de Grado/0. Data
os.chdir(ruta_carpeta)
print("Directorio actual:", os.getcwd())
```

```
Mounted at /content/drive
Directorio actual: /content/drive/MyDrive/PUJ - Maestría - Proyecto de Grado/0
```

```
import tensorflow as tf
print(tf.config.list_physical_devices('GPU'))
print("GPU available:", tf.config.list_physical_devices('GPU'))
```

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
GPU available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU
```

1. Importe de librerias

Se cargan las librerías necesarias para manipulación de datos, procesamiento de imágenes y modelado con redes neuronales. Algunas destacadas:

- Pandas y NumPy: Para manejo de datos.
- Matplotlib y Seaborn: Para visualización.
- OpenCV (cv2) y PIL (Image): Para procesamiento de imágenes.
- TensorFlow y Keras: Para construir y entrenar modelos de deep learning.

```
import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

import os
import zipfile
import shutil
import seaborn as sns
import datetime
import cv2


import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPl
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam


from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

from PIL import Image  # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")
```

```
#Medir_tiempo_ejecucion():
inicio = datetime.datetime.now()
print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")
```

```
Inicio: 2025-04-07 17:43:16.746034
```

2. Importe de Datos

- Se importa un archivo CSV (Data_Final.csv) con el listado de imágenes y etiquetas.
- Se eliminan registros de la clase "Orange".
- Se divide el dataset en 80% entrenamiento - 20% prueba, asegurando balance entre clases con stratify.

```
data=pd.read_csv('Metadatos_Finales.csv', sep=";")
data.head()
```

	Dataset	Carpeta	Nombre del archivo	Planta	Condición	Saludable	Planta_Estad
0	Cassava Leaf Disease Classification	Cassava	1000015157.jpg	Yuca	Cassava Bacterial Blight (CBB)	No	Yuca_si
1	Cassava Leaf Disease Classification	Cassava	1000201771.jpg	Yuca	Cassava Mosaic Disease	No	Yuca_si

```
#Quitar las naranjas
data = data[data["Planta"] != "Orange"]
data = data[data["Planta"] != "Grape"]
data = data[data["Planta"] != "Pepper_bell"]
data = data[data["Planta"] != "Peach"]
```

3. Análisis de Datos y Creación de conjuntos de train & test

- Se analiza la distribución de imágenes por categoría mediante gráficos de conteo. Se visualizan imágenes de plantas saludables y enfermas usando OpenCV.

Se obtiene el tamaño de las imágenes para estandarizar el preprocesamiento.

3.1. Conteo de imágenes por categoría Plantas (Especies)

```
data.Planta.value_counts()
```

Planta	count
Yuca	21397
Tomato	18160
Corn_(maize)	3852
Apple	3174
Potato	2152
Strawberry	1565
bean	1296
cucumber	691
cauliflower	656

dtype: int64

3.2. Division aleatoria entre conjunto de train y test usando una dividion 80% - 20%

```
train, test= train_test_split( data, test_size=0.2, random_state=42,shuffle=True,
train.shape, test.shape
```

```
((42354, 7), (10589, 7))
```

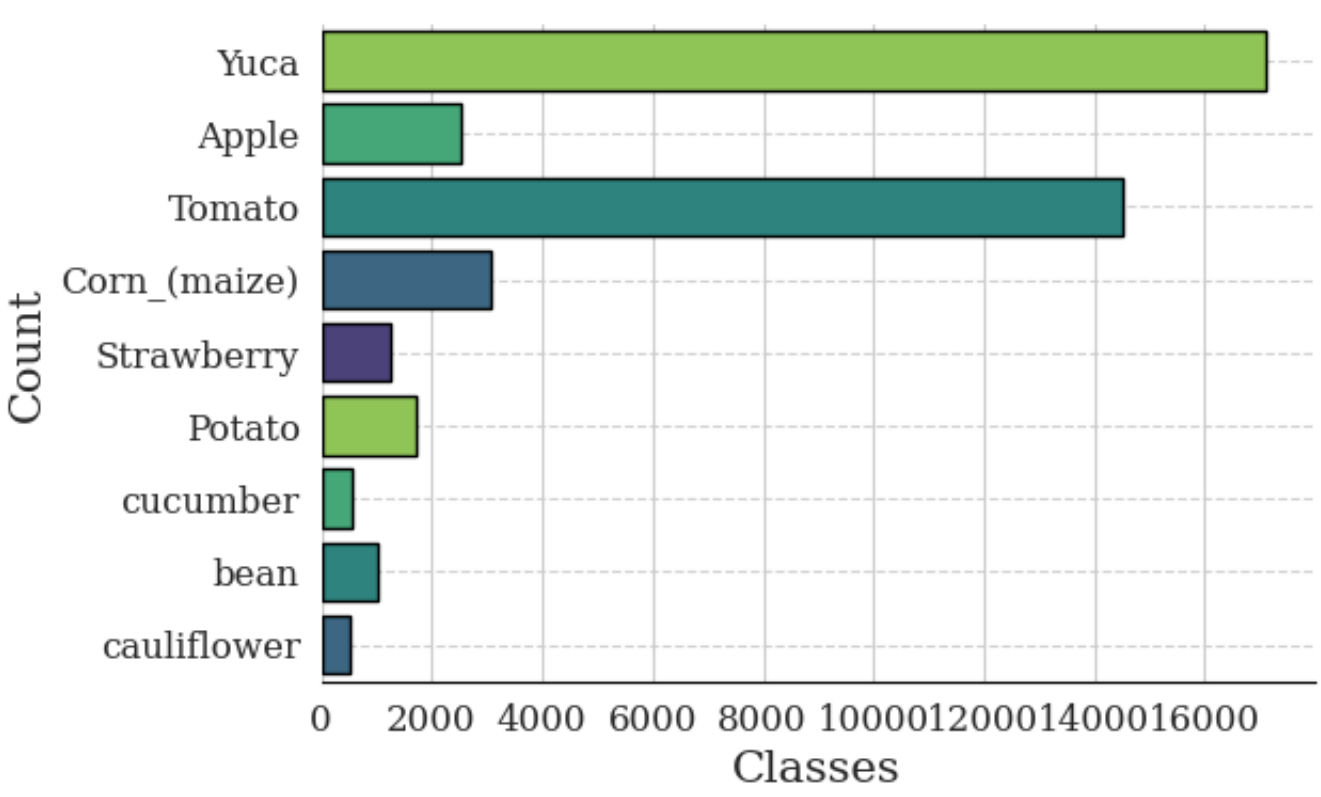
3.3. Conteo de imágenes por en el conjunto train por categoría

```
train.Planta.value_counts()
```

Planta	count
Yuca	17117
Tomato	14528
Corn_(maize)	3081
Apple	2539
Potato	1722
Strawberry	1252
bean	1037
cucumber	553
cauliflower	525

dtype: int64

```
sns.set_style("whitegrid")
fig, ax = plt.subplots(figsize = (6, 4))

for i in ['top', 'right', 'left']:
    ax.spines[i].set_visible(False)
ax.spines['bottom'].set_color('black')

sns.countplot(train.Planta, edgecolor = 'black',
              palette = reversed(sns.color_palette("viridis", 5)))
plt.xlabel('Classes', fontfamily = 'serif', size = 15)
plt.ylabel('Count', fontfamily = 'serif', size = 15)
plt.xticks(fontfamily = 'serif', size = 12)
plt.yticks(fontfamily = 'serif', size = 12)
ax.grid(axis = 'y', linestyle = '--', alpha = 0.9)
plt.show()
```



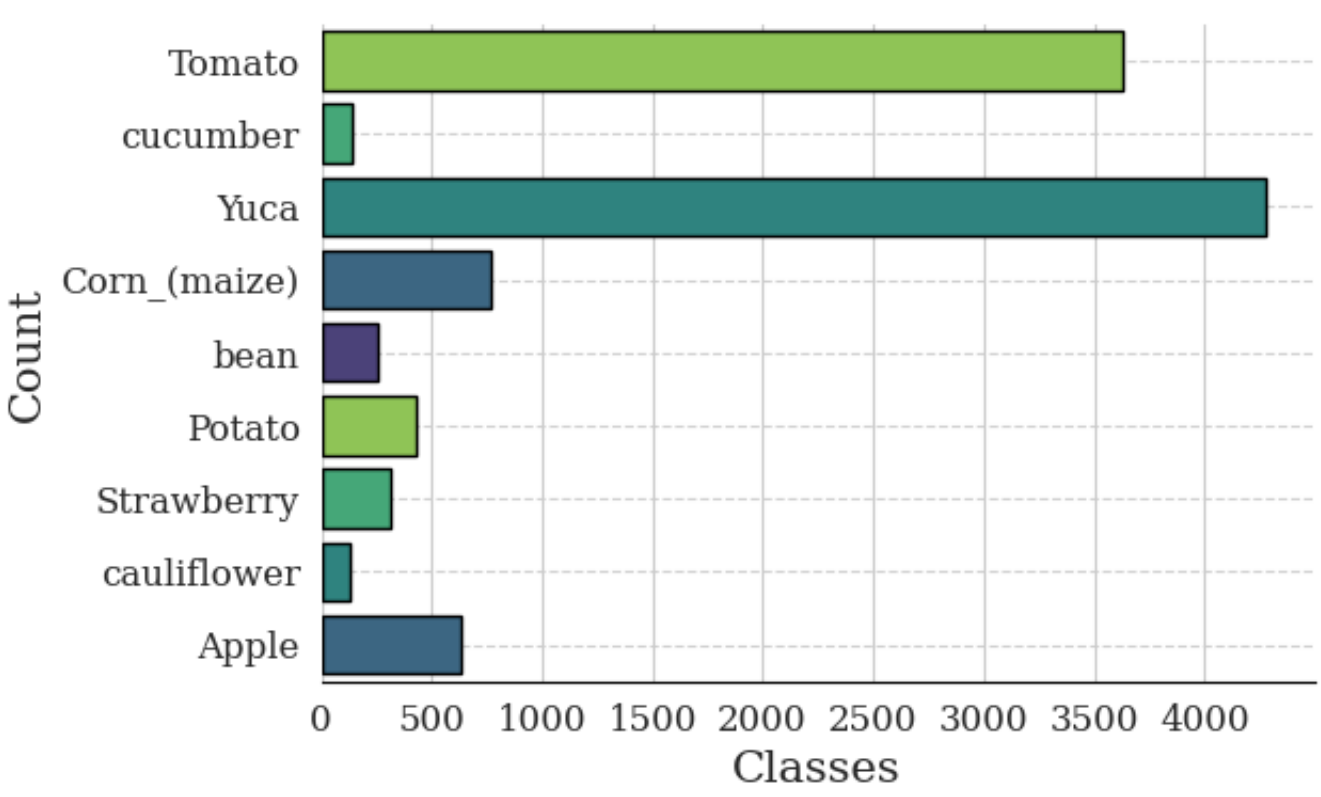
3.4. Conteo de imágenes por en el conjunto test por categoría

```
test.Planta.value_counts()
```

Planta	count
Yuca	4280
Tomato	3632
Corn_(maize)	771
Apple	635
Potato	430
Strawberry	313
bean	259
cucumber	138
cauliflower	131

dtype: int64

```
sns.set_style("whitegrid")
fig, ax = plt.subplots(figsize = (6, 4))

for i in ['top', 'right', 'left']:
    ax.spines[i].set_visible(False)
ax.spines['bottom'].set_color('black')

sns.countplot(test.Planta, edgecolor = 'black',
              palette = reversed(sns.color_palette("viridis", 5)))
plt.xlabel('Classes', fontfamily = 'serif', size = 15)
plt.ylabel('Count', fontfamily = 'serif', size = 15)
plt.xticks(fontfamily = 'serif', size = 12)
plt.yticks(fontfamily = 'serif', size = 12)
ax.grid(axis = 'y', linestyle = '--', alpha = 0.9)
plt.show()
```



3.5. Muestra de Imagenes

```
# Cambiar el nombre de la columna
test = test.rename(columns={'Nombre del archivo': 'image_id'})
train = train.rename(columns={'Nombre del archivo': 'image_id'})
```

```
train.head()
```

	Dataset	Carpeta	image_id
1580	Cassava Leaf Disease Classification	Cassava	1277648239.jpg
52778	PlantVillage	Apple___Apple_scab	3c27bbd7-b305-4ab0-8a85-5061363cf632___FREC_Sc...
45528	PlantVillage	Tomato___healthy	87f08e15-5b2d-4d01-b83b-fd958eba02d4___GH_HL L...

4. Preparación del Modelo

- Se implementará data augmentation para mejorar la generalización.
- Se evaluará el modelo con métricas como precisión y recall.

```
# Hiperparámetros principales
BATCH_SIZE = 64
STEPS_PER_EPOCH = len(train)*0.8 / BATCH_SIZE
VALIDATION_STEPS = len(train)*0.2 / BATCH_SIZE
EPOCHS = 10
TARGET_SIZE = 150
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Asegura que las etiquetas sean strings
train.Planta = train.Planta.astype(str)

# Fijamos semilla para reproducibilidad
SEED = 42

# Generator para entrenamiento con data augmentation + normalización
train_datagen = ImageDataGenerator(
    validation_split=0.2,
```

```
    rescale=1./255,
    rotation_range=45,
    zoom_range=0.2,
    horizontal_flip=True,
    vertical_flip=True,
    fill_mode='nearest',
    shear_range=0.1,
    height_shift_range=0.1,
    width_shift_range=0.1
)

train_generator = train_datagen.flow_from_dataframe(
    dataframe=train,
    directory="Datos Finales",
    x_col="image_id",
    y_col="Planta",
    subset="training",
    target_size=(TARGET_SIZE, TARGET_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    shuffle=True,
    seed=SEED,
    interpolation='bilinear'
)

# Generator para validación: solo normalización, sin data augmentation
validation_datagen = ImageDataGenerator(
    validation_split=0.2,
    rescale=1./255
)

validation_generator = validation_datagen.flow_from_dataframe(
    dataframe=train,
    directory="Datos Finales",
    x_col="image_id",
    y_col="Planta",
    subset="validation",
    target_size=(TARGET_SIZE, TARGET_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    shuffle=False,
    seed=SEED,
    interpolation='bilinear'
)
```

```
Found 33612 validated image filenames belonging to 9 classes.
Found 8403 validated image filenames belonging to 9 classes.
```

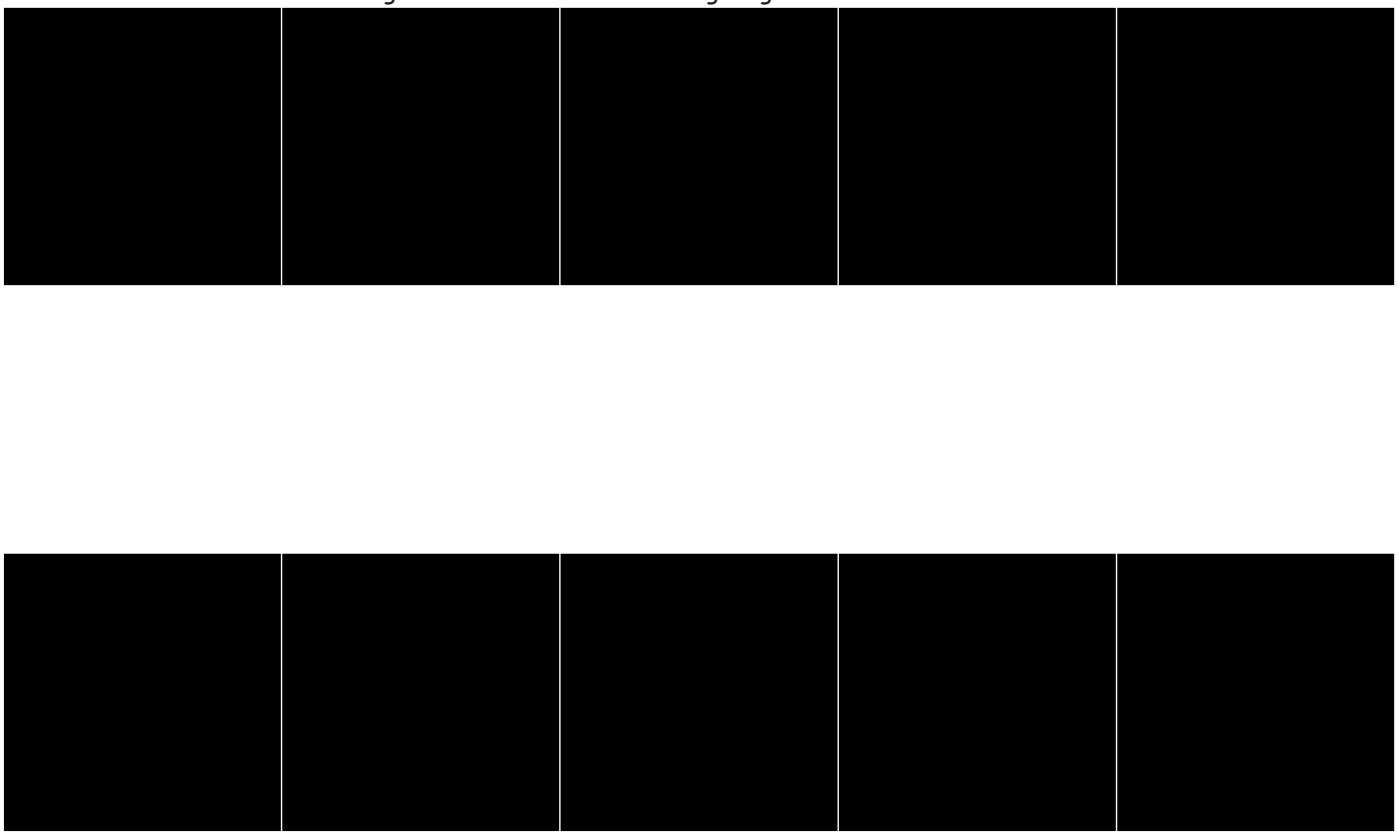
Imágen Aleatoria antes de Augmentation

```
img_path = os.path.join("Datos Finales", train.image_id[20])
img = image.load_img(img_path, target_size = (TARGET_SIZE, TARGET_SIZE))
img_tensor = image.img_to_array(img)
img_tensor = np.expand_dims(img_tensor, axis = 0)
img_tensor /= 255.

plt.imshow(img_tensor[0])
plt.axis('off')
plt.show()
```

```
generator = train_datagen.flow_from_dataframe(train.iloc[20:21],
                          directory = "Datos Finales",
                          x_col = "image_id",
                          y_col = "Planta",
                          target_size = (TARGET_SIZE, TARGET_SIZE),
                          batch_size = BATCH_SIZE,
```

```
                            class_mode = "categorical")

aug_images = [generator[0][0][0]/255 for i in range(10)]
fig, axes = plt.subplots(2, 5, figsize = (TARGET_SIZE, TARGET_SIZE))
axes = axes.flatten()
for img, ax in zip(aug_images, axes):
    ax.imshow(img)
    ax.axis('off')
plt.tight_layout()
plt.show()
```

```
Found 1 validated image filenames belonging to 1 classes.
```



5. Construcción del Modelo

5.1. Clasificador

Arquitectura del Modelo CNN El modelo es una red neuronal convolucional (**CNN**) diseñada para clasificar imágenes de plantas en distintas categorías. Su arquitectura incluye:

1. Capas Convolucionales y de MaxPooling Cada capa convolucional extrae características de las imágenes, mientras que **MaxPooling** reduce la dimensionalidad, mejorando la eficiencia del modelo.

- **Capa 1:**
  - `Conv2D(32, (3,3), activation='relu', input_shape=(150,150,3))`
  - `BatchNormalization()`
  - `MaxPooling2D(2,2)`
- **Capa 2:**
  - `Conv2D(64, (3,3), activation='relu')`
  - `BatchNormalization()`
  - `MaxPooling2D(2,2)`
- **Capa 3:**

    - `Conv2D(128, (3,3), activation='relu')`
    - `BatchNormalization()`
    - `MaxPooling2D(2,2)`
- **Capa 4:**
    - `Conv2D(128, (3,3), activation='relu')`
    - `BatchNormalization()`
    - `MaxPooling2D(2,2)`

2. Capa de Aplanamiento Después de las capas convolucionales, se usa `Flatten()` para convertir la salida en un vector de una dimensión.

3. Capas Densas y Regularización

- Una capa densa con **512 neuronas** y función de activación ReLU.
- Se usa **Dropout (0.5)** para reducir el sobreajuste.

4. Capa de Salida

- `Dense(9, activation='softmax')`
- La activación **softmax** se usa porque se trata de un problema de clasificación multiclase (9 categorías).

```
# Arquitectura del modelo CNN
model = models.Sequential()

# Capa 1: Convolucional y MaxPooling
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(2, 2))

# Capa 2: Convolucional y MaxPooling
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(2, 2))

# Capa 3: Convolucional y MaxPooling
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(2, 2))

# Capa 4: Convolucional y MaxPooling
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.MaxPooling2D(2, 2))

# Aplanamiento de la capa convolucional
model.add(layers.Flatten())

# Clasificador
model.add(layers.Dense(512, activation='relu'))
model.add(layers.Dropout(0.5))  # Dropout para evitar el overfitting

# Capa de salida
model.add(layers.Dense(9, activation='softmax'))
```

5.2. Compilación del modelo

```
model.compile(
    loss='categorical_crossentropy',  # Cambiado a 'categorical_crossentropy' para
    optimizer='adam',
    metrics=['accuracy']
)
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	
conv2d (Conv2D)	(None, 148, 148, 32)	
batch_normalization (BatchNormalization)	(None, 148, 148, 32)	
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	
conv2d_1 (Conv2D)	(None, 72, 72, 64)	
batch_normalization_1 (BatchNormalization)	(None, 72, 72, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	
conv2d_2 (Conv2D)	(None, 34, 34, 128)	
batch_normalization_2 (BatchNormalization)	(None, 34, 34, 128)	
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	
conv2d_3 (Conv2D)	(None, 15, 15, 128)	
batch_normalization_3 (BatchNormalization)	(None, 15, 15, 128)	
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 128)	
flatten (Flatten)	(None, 6272)	
dense (Dense)	(None, 512)	3
dropout (Dropout)	(None, 512)	
dense_1 (Dense)	(None, 9)	

Total params: 3,458,633 (13.19 MB)
Trainable params: 3,457,929 (13.19 MB)
Non-trainable params: 704 (2.75 KB)

6. Ejecución del modelo

- Se entrenará el modelo utilizando Adam como optimizador y EarlyStopping para prevenir sobreajuste.
- Se evaluará el desempeño sobre el conjunto de prueba.

```
early_stopping = EarlyStopping(
    monitor='val_loss',  # Monitor the validation loss
    patience=100,  # Number of epochs with no improvement after which training wi
    restore_best_weights=True  # Restore the model weights from the epoch with th
)

from tensorflow.keras.callbacks import ReduceLROnPlateau

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.2,  # Reduce learning rate by a factor of 0.2
    patience=3,  # Number of epochs with no improvement after which learning rate
    min_lr=1e-5  # Minimum learning rate
)
```

```
history = model.fit(
    train_generator,
    epochs=EPOCHS,
    validation_data=validation_generator,
    callbacks=[early_stopping, reduce_lr],
    verbose=2,                    # Puedes poner 1 o 0 si quieres que imprima men
    )
```

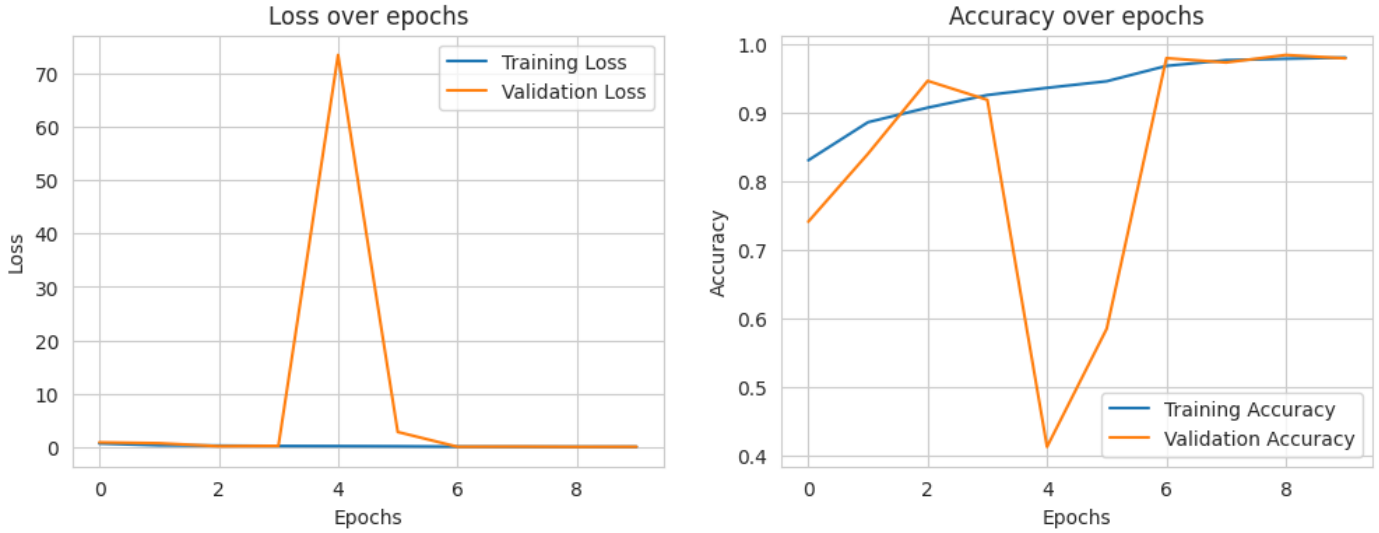
```
Epoch 1/10
526/526 - 10669s - 20s/step - accuracy: 0.8298 - loss: 0.6784 - val_accuracy:
Epoch 2/10
526/526 - 420s - 798ms/step - accuracy: 0.8853 - loss: 0.3411 - val_accuracy:
Epoch 3/10
526/526 - 420s - 799ms/step - accuracy: 0.9065 - loss: 0.2775 - val_accuracy:
Epoch 4/10
526/526 - 418s - 794ms/step - accuracy: 0.9249 - loss: 0.2223 - val_accuracy:
Epoch 5/10
526/526 - 420s - 798ms/step - accuracy: 0.9354 - loss: 0.2000 - val_accuracy:
Epoch 6/10
526/526 - 419s - 797ms/step - accuracy: 0.9450 - loss: 0.1664 - val_accuracy:
Epoch 7/10
526/526 - 419s - 797ms/step - accuracy: 0.9674 - loss: 0.0956 - val_accuracy:
Epoch 8/10
526/526 - 416s - 790ms/step - accuracy: 0.9758 - loss: 0.0750 - val_accuracy:
Epoch 9/10
526/526 - 418s - 795ms/step - accuracy: 0.9779 - loss: 0.0660 - val_accuracy:
Epoch 10/10
526/526 - 417s - 793ms/step - accuracy: 0.9795 - loss: 0.0612 - val_accuracy:
```

6.1. Resultados del Modelo

```
# Acceder a los datos de pérdida y precisión
loss = history.history['loss']
val_loss = history.history['val_loss']
accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

# Graficar la pérdida del modelo
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss over epochs')
plt.xlabel('Epochs')
```

```
plt.ylabel('Loss')
plt.legend()

# Graficar la precisión del modelo
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy over epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7adf809657d0>
```



```
epochs = [1, 2, 3, 4, 5, 6, 7, 8,9,10]
data = {
    'Epoch': epochs,
    'Trainning Loss': loss,
    'Validation Loss': val_loss,
    'Training Accuracy': accuracy,
    'Validation  Accuracy': val_accuracy
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)
```

```
   Epoch  Trainning Loss  Validation Loss  Training Accuracy  \
0      1        0.678399         0.894565           0.829823
1      2        0.341134         0.738363           0.885309
2      3        0.277498         0.171599           0.906521
3      4        0.222276         0.235196           0.924938
4      5        0.200007        73.430382           0.935410
5      6        0.166443         2.840817           0.945049
6      7        0.095642         0.074032           0.967422
7      8        0.075032         0.098644           0.975753
8      9        0.065961         0.055390           0.977865
9     10        0.061189         0.073147           0.979531

   Validation  Accuracy
0              0.740331
1              0.839343
2              0.945615
3              0.917886
4              0.411996
5              0.584196
6              0.978698
7              0.972391
8              0.983339
9              0.978460
```

```
print('Perdida final en Train:', history.history['loss'][-1])
print("Pérdida final en Validation:", history.history['val_loss'][-1])


print("Accuracy final en Train:", history.history['accuracy'][-1])
print("Accuracy final en Validation:", history.history['val_accuracy'][-1])
```

```
Perdida final en Train: 0.061189353466033936
Pérdida final en Validation: 0.07314743101596832
Accuracy final en Train: 0.9795311093330383
Accuracy final en Validation: 0.9784600734710693
```

```
fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

```
Fin: 2025-04-07 21:48:20.468077
Duración: 4:05:03.722043
```

7. Evaluación del Modelo

7.1. Lote de imagenes de testeo

```
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisp
```

```
test.Planta = test.Planta.astype('str')

test_generator =  train_datagen.flow_from_dataframe(test,
                        directory = "Datos Finales",
                        x_col = "image_id",
                        y_col = "Planta",
                        target_size = (TARGET_SIZE, TARGET_SIZE),
                        batch_size = BATCH_SIZE,
                        class_mode = "categorical")
```

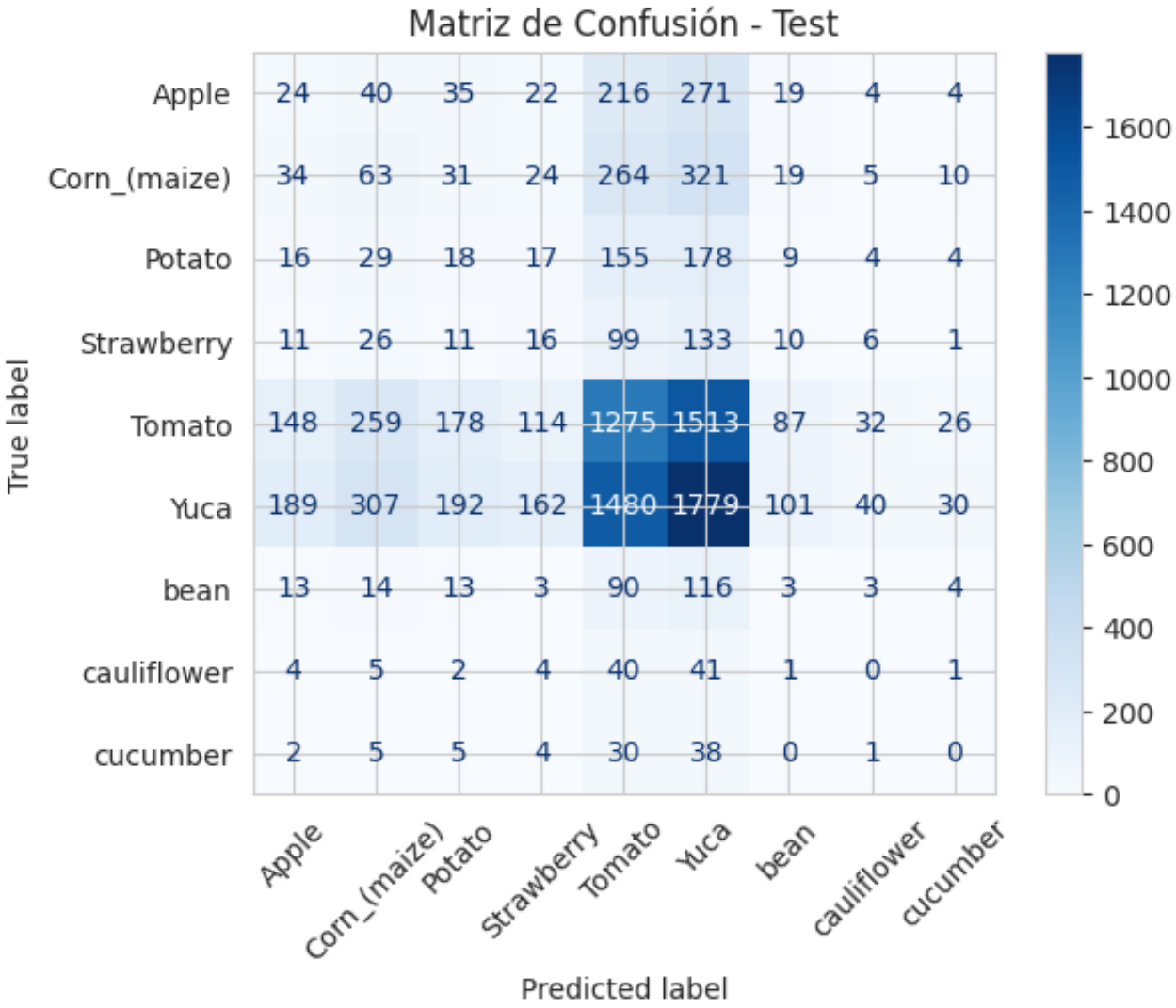
```
Found 10503 validated image filenames belonging to 9 classes.
```

7.2. Predicciones con imagenes de Testeo

```
# ------------------------
# 1. Obtener predicciones completas para test
# ------------------------
y_pred_probs = model.predict(test_generator)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = test_generator.classes
```

```
165/165 ━━━━━━━━━━━━━━━━━━━━ 2922s 18s/step
```

```
# ------------------------
# 2. Matriz de Confusión
# ------------------------
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_generator.
disp.plot(cmap="Blues", xticks_rotation=45)
plt.title("Matriz de Confusión - Test")
plt.show()
```



7.3. Presición de la Predicción

```
accuracy = accuracy_score(y_true, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

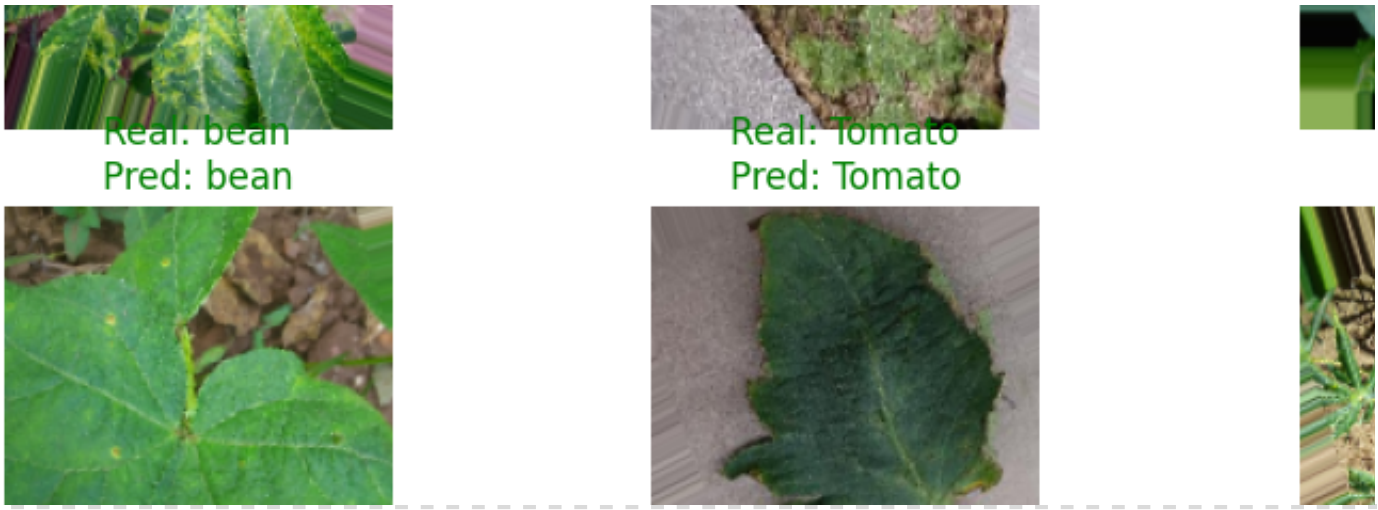
```
Test Accuracy: 0.3026
```

7.4. Muestreo visual de predicciones vs etiquetas reales

```
# Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())

# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i]]
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
```

```
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y=1.02)
plt.show()
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 2s 21ms/step
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-34-d11e7bf3f5b8> in <cell line: 0>()
     26 plt.figure(figsize=(15, 8))
     27 for i in range(num_samples):
---> 28     plt.subplot(3, 4, i + 1)
     29     img_to_show = normalize_for_display(x_batch[i])
     30     plt.imshow(img_to_show)

                                 1 frames
/usr/local/lib/python3.11/dist-packages/matplotlib/gridspec.py in
_from_subplot_args(figure, args)
    587         else:
    588             if not isinstance(num, Integral) or num < 1 or num >
rows*cols:
--> 589                 raise ValueError(
    590                     f"num must be an integer with 1 <= num <=
{rows*cols}, "
    591                     f"not {num!r}"

ValueError: num must be an integer with 1 <= num <= 12, not 13
```

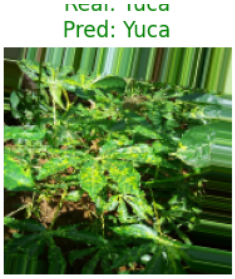


Próximos pasos: Explicar error

```
# Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())

# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
```

```
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y=1.02
plt.show()
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-35-d11e7bf3f5b8> in <cell line: 0>()
     26 plt.figure(figsize=(15, 8))
     27 for i in range(num_samples):
---> 28     plt.subplot(3, 4, i + 1)
     29     img_to_show = normalize_for_display(x_batch[i])
     30     plt.imshow(img_to_show)

                                 1 frames
/usr/local/lib/python3.11/dist-packages/matplotlib/gridspec.py in
_from_subplot_args(figure, args)
    587         else:
    588             if not isinstance(num, Integral) or num < 1 or num >
rows*cols:
--> 589                 raise ValueError(
    590                     f"num must be an integer with 1 <= num <=
{rows*cols}, "
    591                     f"not {num!r}"

ValueError: num must be an integer with 1 <= num <= 12, not 13
```

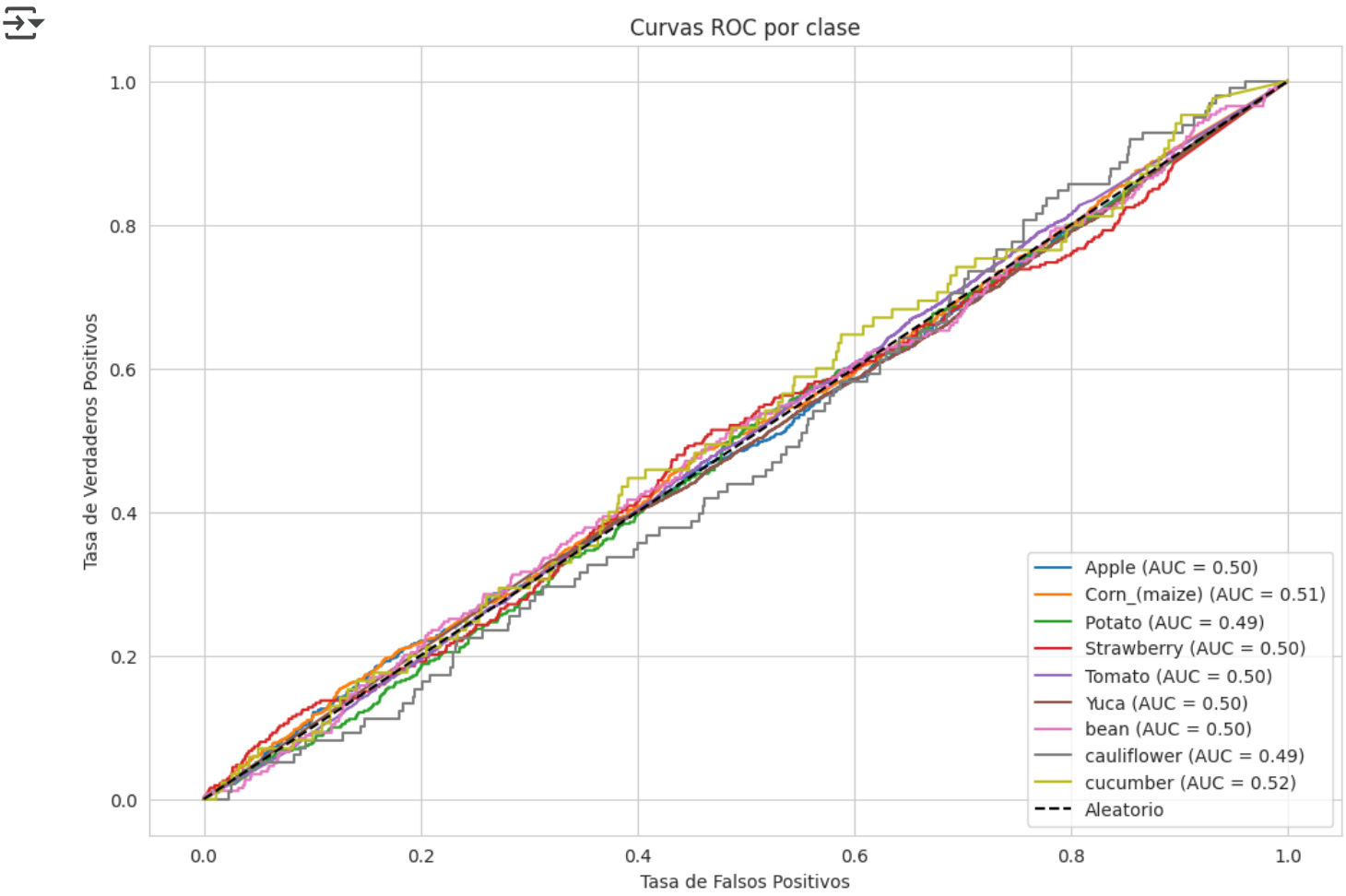


Próximos pasos: Explicar error

7.5. Curva ROC

```
from sklearn.metrics import roc_curve, auc
from tensorflow.keras.utils import to_categorical

# 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())
```

```
# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```


Curvas ROC por clase
Tasa de Verdaderos Positivos
Tasa de Falsos Positivos
0.0
0.2
0.4
0.6
0.8
1.0
Apple (AUC = 0.50)
Corn_(maize) (AUC = 0.51)
Potato (AUC = 0.49)
Strawberry (AUC = 0.50)
Tomato (AUC = 0.50)
Yuca (AUC = 0.50)
bean (AUC = 0.50)
cauliflower (AUC = 0.49)
cucumber (AUC = 0.52)
Aleatorio

```
from sklearn.metrics import roc_auc_score

# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

```
AUC Macro promedio: 0.5007
AUC Micro promedio: 0.6357
```

7.6. Resultado de la Evaluación del Modelo

```
from sklearn.metrics import accuracy_score, f1_score, classification_report

# Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes

# Accuracy
accuracy = accuracy_score(y_true, y_pred)

# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))
```

```
Accuracy: 0.3026
F1 Score (macro): 0.1096
F1 Score (micro): 0.3026
F1 Score (weighted): 0.3004

Reporte de clasificación por clase:
              precision    recall  f1-score   support

       Apple       0.05      0.04      0.04       635
Corn_(maize)       0.08      0.08      0.08       771
      Potato       0.04      0.04      0.04       430
  Strawberry       0.04      0.05      0.05       313
      Tomato       0.35      0.35      0.35      3632
        Yuca       0.41      0.42      0.41      4280
        bean       0.01      0.01      0.01       259
 cauliflower       0.00      0.00      0.00        98
    cucumber       0.00      0.00      0.00        85

    accuracy                           0.30     10503
   macro avg       0.11      0.11      0.11     10503
weighted avg       0.30      0.30      0.30     10503
```